CPSC 375
Introduction to Data Science and Big Data Analytics

Dr. Kanika Sood

kasood@fullerton.edu

What we will cover this week

- k-nearest neighbors
  - In Python
- Distance metrics

K-nearest neighbors algorithm

CLASSIFICATION

- Write a function that will predict if a name is masculine or feminine

Input	Output
Laura	F
Carlos	
Jose	
Maria	
Belen	

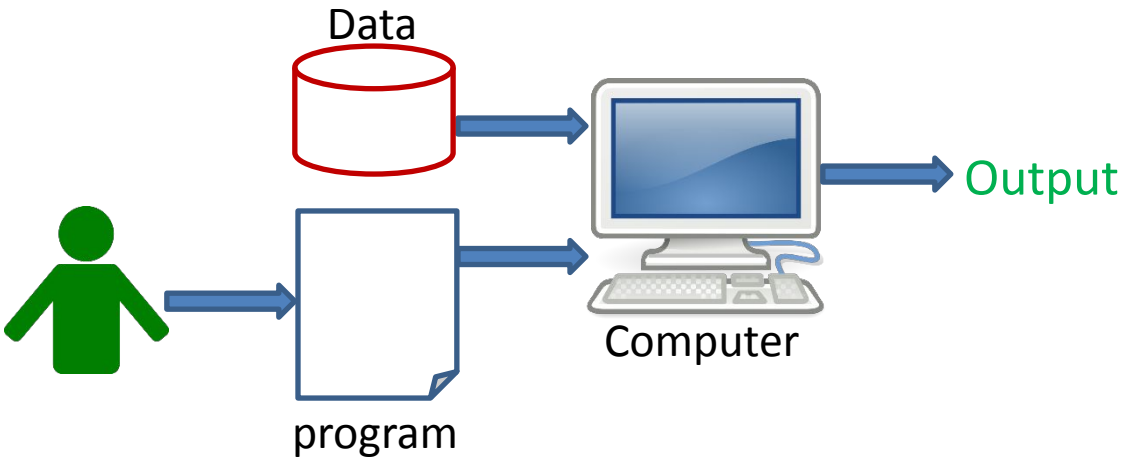
Write a function to guess if a name is typically masculine/feminine

```
def m_or_f(name):

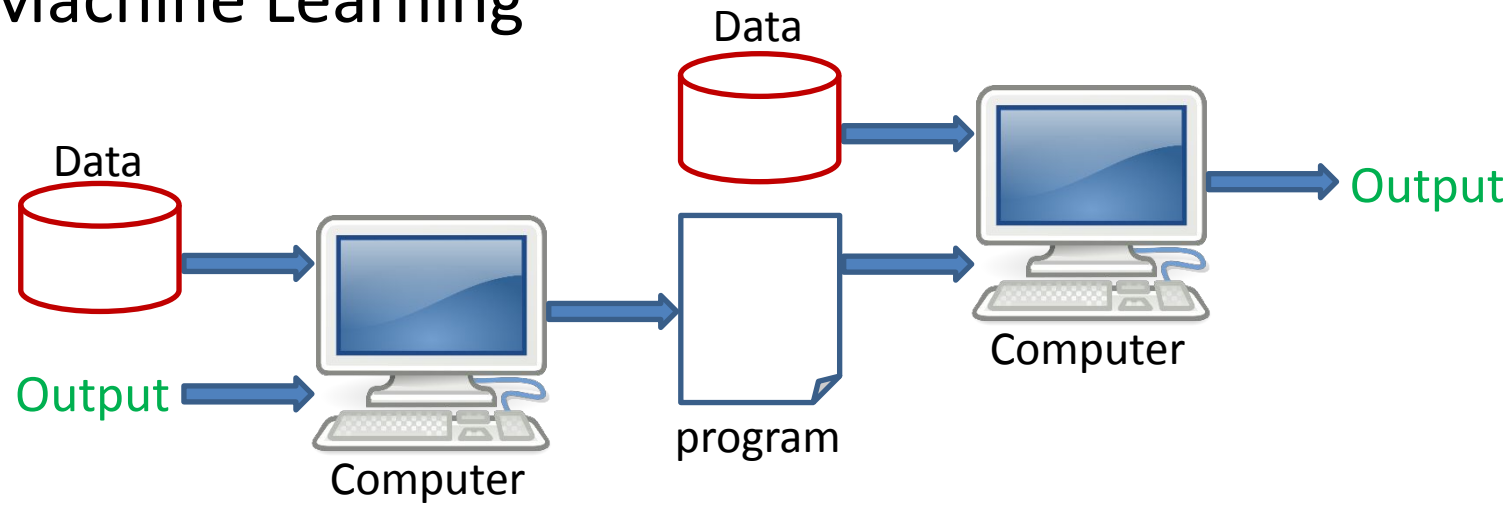

    ??
```

```
def m_or_f(name):
  if name[-1] == "a":
    return "F"
  else:
    return "M"
```

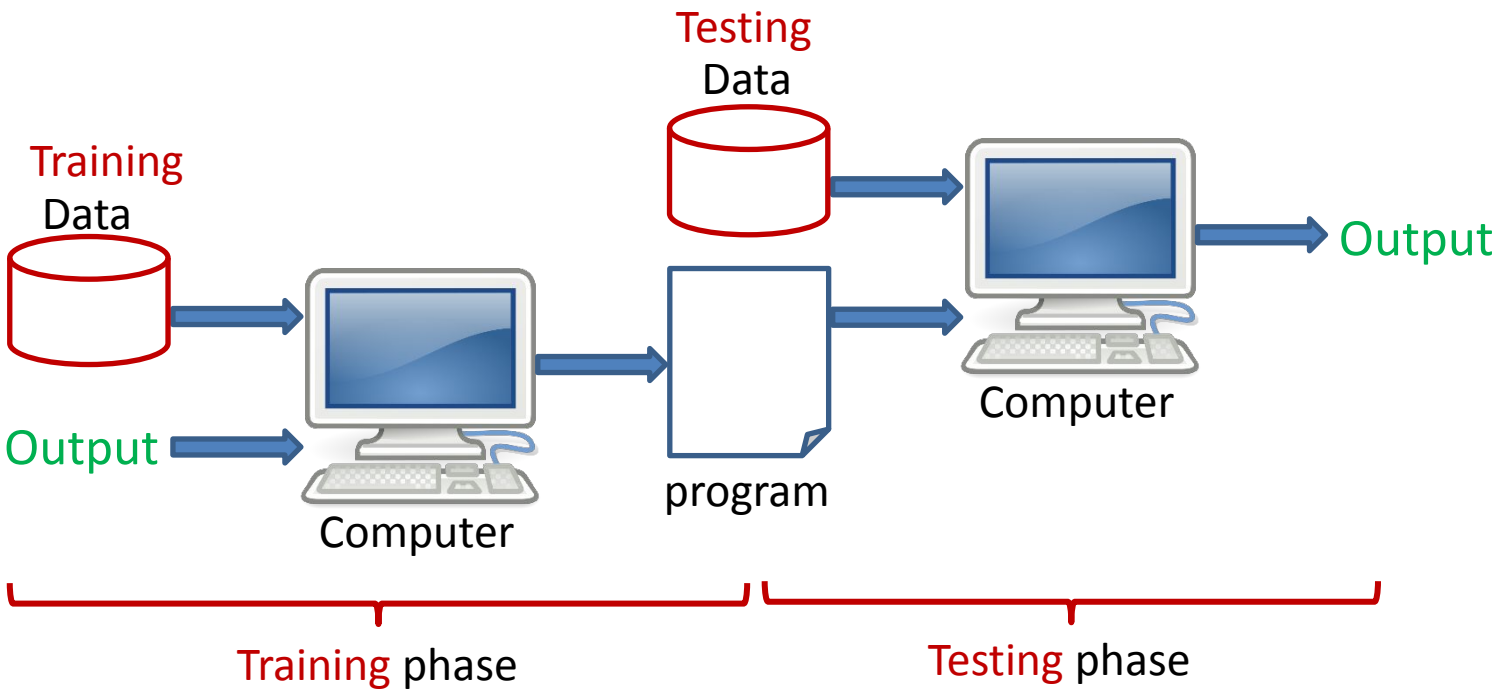
Traditional Programming



Machine Learning



Machine Learning



The machine learning approach

```
def m_or_f (name):
```

mydata =

Input	Output
Laura	F
Carlos	M
Jose	M
Maria	F
Belen	F

// compare name with mydata.Input and find "closest" match – the *nearest neighbor –* and return corresponding output

Structure of training and test data

Input	Output
Laura	F
Carlos	M
Jose	M
Maria	F
Belen	F



Iris setosa



Iris versicolor



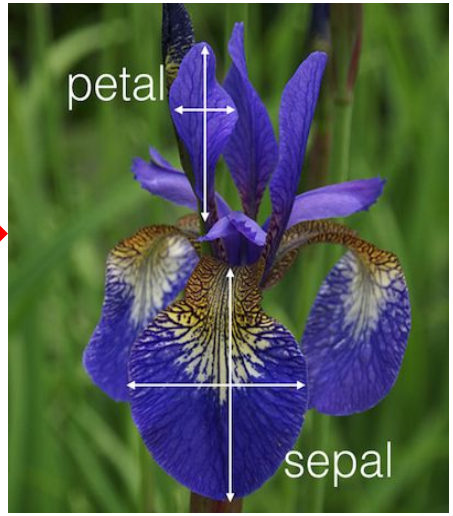
Iris virginica

Input	nchars	lastchar	nvowels	Output
Laura	5	‘a’	3	F
Carlos	6	‘s’	2	M
Jose	4	‘e’	2	M
Maria	5	‘a’	3	F
Belen	5	‘n’	2	F

Features/Feature vector

Label/ Class





Sepal. Length	Sepal. Width	Petal. Length	Petal. Width	Output
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.3	3.3	6.0	2.5	virginica
…	…	…	…	…

Supervised Learning

Features

Labels/Classes

Petal Length	Petal Width	Sepal Length	Sepal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.2	2.9	4.3	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.1	2.5	3.0	1.1	Versicolor
7.1	3.0	5.9	2.1	Virginica

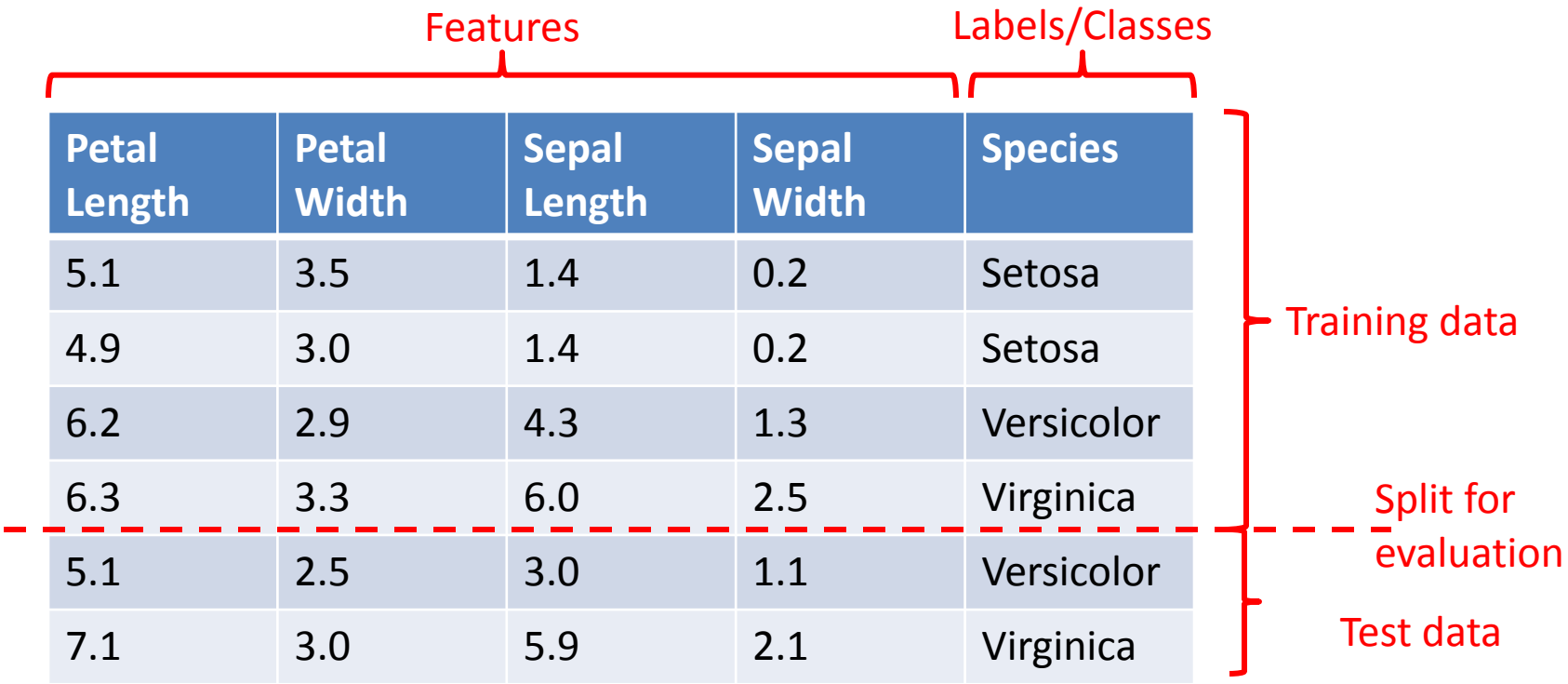
an *instance*

Training data

5.1	3.5	1.4	0.2	?
7.1	3.0	5.9	2.1	?

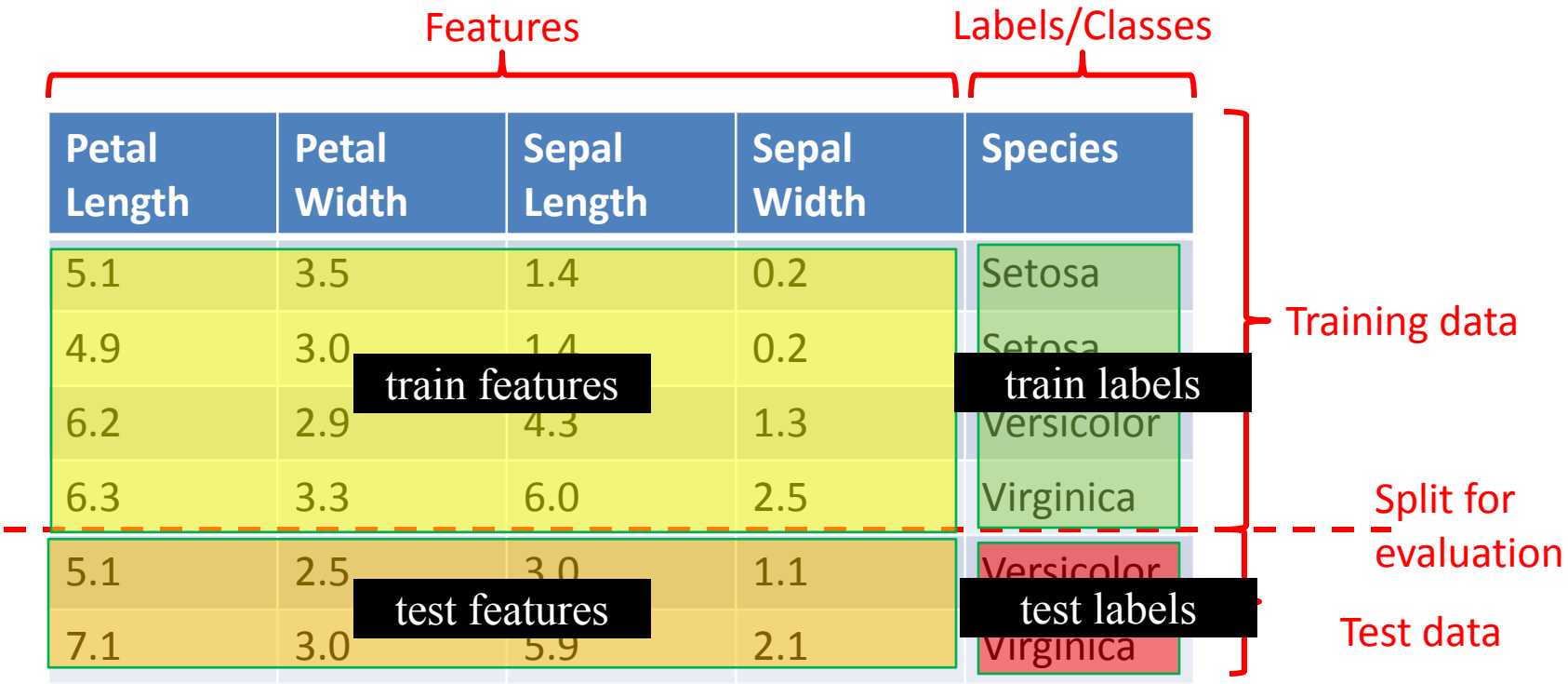
Predictions

Supervised Learning



Petal Length	Petal Width	Sepal Length	Sepal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.2	2.9	4.3	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.1	2.5	3.0	1.1	Versicolor
7.1	3.0	5.9	2.1	Virginica

Supervised Learning



Is linear regression machine learning?

- Yes!
- After a model is obtained, it can be used to predict given new independent values
- Model-based learning
  - Model built on assumptions of the data

Is linear regression machine learning?

- Yes!
  - The program is

```
def predict(x):
    y = β0 + β1 x;
    return y
```

- Supervised learning
  - Training algorithm: get model parameters β_0, β_1 using least squares estimation
    - mod = LinearRegression()
    - mod.fit(X_train, y_train)
  - Testing algorithm: compute $y = \beta_0 + \beta_1 x$
    - mod.predict(X_test)

Model-based learning

- Linear regression
  - Model is a linear equation
- Decision trees
  - Model is a decision tree
- Bayesian networks
  - Model uses probabilities
- Neural networks
  - Model is a network of artificial neurons
- …

Instance-based learning

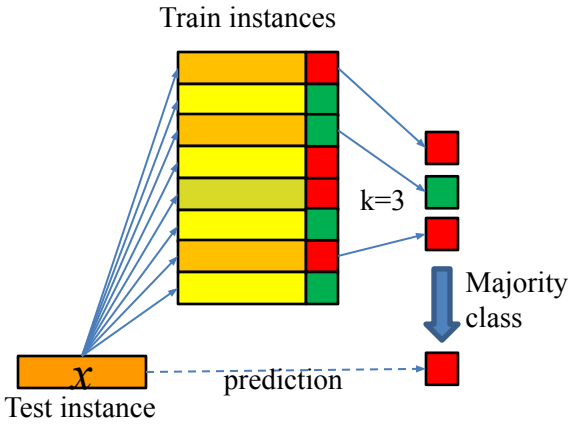
- What if we don't know the model structure?
  - Linear, quadratic, …?
- Instance-based learning
  - No explicit model, predict directly from data (*instances*)
- Learning is just storing all the training data
- When a *new* instance is given:
  - Get from stored instances a set of similar instances
  - Use the classes of the similar instances to classify the new instance

k-Nearest Neighbor (k-NN) algorithm

- Simplest instance-based method
- Other approaches
  - Support Vector Machines
  - Vector quantization

k-Nearest Neighbor classification

- Training algorithm :
  - Save all the training instances
  - Each instance is a pair:
    - (feature vector, output class)
- To predict class of x (Classification algorithm):
  - Find the *k* training instances that are closest to x
  - Predicted: majority class of these *k* examples



“Closest” in vector space

- Every instance feature is a *d*-dimensional vector
- $\vec{x} = (x_1, x_2, \ldots, x_d)$
- Distance between two data points, $\vec{x}$ and $\vec{y}$, is calculated using Euclidean distance
- $\vec{x} = (x_1, x_2, \ldots, x_d)$
- $\vec{y} = (y_1, y_2, \ldots, y_d)$

$$\|\vec{x} - \vec{y}\| = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

Classwork

- soodk6/cpsc375-classwork
  - knn.ipynb – Part 1

k-NN in scikit-learn

- `from sklearn.neighbors import KNeighborsClassifier`
- `mod = KNeighborsClassifier(n_neighbors=3)`
- `mod.fit(<train data>, <train labels>)`
- `mod.predict(<test data>)`
- `mod.score(<test data>, <test labels>)`

How to determine the value for k?

- Determined experimentally
- Split data into two: train and test sets
- Start with k=1 and use the test set to validate the error rate of the classifier
- Repeat with k=k+1
- Choose the value of k for which the error rate is minimum
  - Look for the “elbow” in the error vs. k curve

- Note: if there are only 2 classes, choosing k to be odd (1,3,5,…) avoids ties

How to determine the value for k?

- A "rule-of-thumb"
- Choose $\mathrm{k} = \sqrt{n}$ where n is number of instances
  - For iris, n = ?
  - k =

How to determine the value for k?

- A "rule-of-thumb"
- Choose $\mathrm{k} = \sqrt{n}$ where n is number of instances
  - For iris, n = ?
  - k =
- Why?
  - k too small: susceptible to errors
  - k too large: saying that answer depends not just on the nearest neighbors (k-NN is not a good choice)

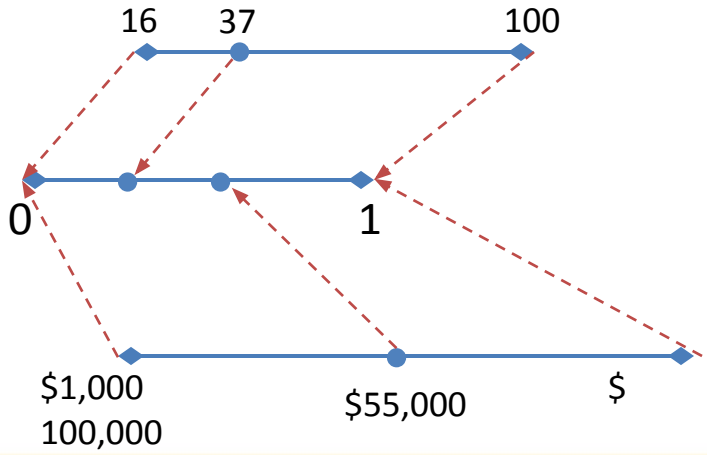
Classwork

- soodk6/cpsc375-classwork
  - knn.ipynb – Part 2

Normalization

- Units matter!
- Large valued features dominate the distance calculation
- Normalization:
  - Convert each feature independently into comparable ranges (e.g., 0…1)
  - Percentile:
    - Proportion of the Min to Max range
  - Example:

$$\frac{37 - min}{max - min} = \frac{37 - 16}{100 - 16} = 0.25$$



Continuous-valued target functions

- Calculate the mean value of the *k* nearest training examples
  - Instead of their most common value
  - "Nearest Neighbors Regression" / `KNeighborsRegressor`

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} f(x_i)}{k}$$

Curse of Dimensionality

- Imagine instances described by 20 features
  - BUT only 3 are relevant to target class
- Curse of dimensionality:
  - nearest neighbor is misled when instance space is high-dimensional
  - Dominated by large number of irrelevant features

- At very large number of dimensions
  - Euclidean distance itself becomes less useful
  - Distances between all pairs of points start taking on similar values
  - Can no longer use the intuition that closeness means correctness

Dimensionality reduction

- Remove variables from the dataset that are not relevant to the class being predicted
- How to identify such variables?
  - Can use linear regression
  - Fit a linear model between predicted class and each variable (*one at a time*)
  - Prioritize variables in order of their linear model fit
- Will not always work
  - Two variables may not be correlated with the class on their own
  - But are important when taken together

When to use k-Nearest Neighbors

- Instances map to points in a vector space
- Lots of training data

Advantages of k-Nearest Neighbors

- Training is very fast
  - Just store the instances
- Learn complex target functions

Disadvantages of k-Nearest Neighbors

- Slow at query time
  - Nearly all computation takes place at classification time rather than learning time
  - Presorting and indexing training samples into search trees reduces time
- Starts failing with increasing dimensions

Consider this data set

X	Y
2	EVEN
11	ODD
23	ODD
8	EVEN
18	EVEN
7	ODD
3	?

What went wrong?

- The distance metric
  - Used Euclidean distance
  - Should have used parity

Distance metrics

- Distance metric:
  - Formula defining the distance between two points in the vector space
- Most common: Euclidean distance
  - Square root of the sum of squared differences of each component

$$\|\vec{x} - \vec{y}\| = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

- Select distance metric appropriate for the problem

Distance metrics

- Manhattan distance
- Jaccard similarity
- Cosine similarity
- Edit distance or Hamming distance
- A custom distance metric

Manhattan distance

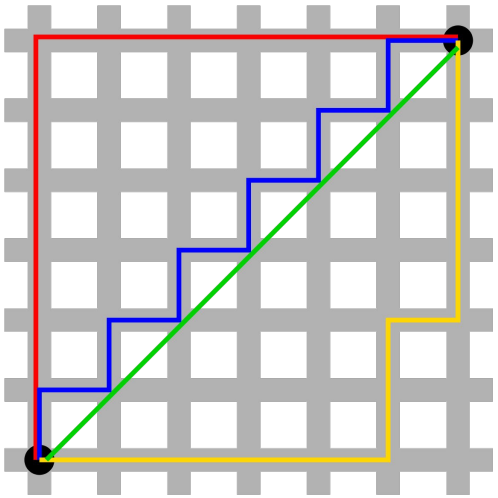
- Euclidean distance:
  - L2 norm
- Manhattan distance:
  - L1 norm

$$d(\vec{x}, \vec{y}) = \sum_{i=1}^{k} |x_i - y_i|$$

Manhattan distance

- Euclidean distance:
  - "As the crow flies"
- Manhattan distance:
  - "Walking only on streets"

$$d(\vec{x},\vec{y}) = \sum_{i=1}^{k} |x_i - y_i|$$





Manhattan distance

- What is the Manhattan distance between:
  - (5,65) and (4, 70)?
  - (5,65, 2) and (4, 70, -1)?

- What is the Euclidean distance between:
  - (5,65) and (4, 70)?
  - (5,65, 2) and (4, 70, -1)?

Manhattan distance

- What is the Manhattan distance between:
  - (5,65) and (4, 70)?
  - |5-4| + |65-70| = 1 + 5 = 6
  - (5,65, 2) and (4, 70, -1)?
  - |5-4| + |65-70| + |2 – (-1)| = 9
- What is the Euclidean distance between:
  - (5,65) and (4, 70)?
  - $sqrt((5-4)^2 + (65-70)^2)$ = $\sqrt{26}$= 5.0…

Jaccard similarity

- Similarity between sets

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- A = {Apple, Orange, Banana}
- B = {Apple, Grape}
- J(A,B) = ?
- Max value = ?
- Min value = ?

Jaccard similarity

- Similarity between sets

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- A = {Apple, Orange, Banana}
- B = {Apple, Grape}
- J(A,B) = |{Apple}|/|{Apple,Orange,Banana,Grape}|
  - =1/4 = 0.25
- Max value = 1 (when sets are exactly the same)
- Min value = 0 (when there are no common elements)

Class work

- soodk6/cpsc375-classwork
  - knn.ipynb – Part 3

Use 1-NN algorithm to predict

A	{Apple, Banana, Soymilk, Yoghurt}	Vegetarian
B	{Apple, Peanuts, Yoghurt}	Vegetarian
C	{Tomatoes, Potatoes, Yoghurt}	Meat-eater
D	{Apples,Tomatoes, Potatoes}	Meat-eater
E	{Apples, Tomatoes}	?
F	{Yoghurt}	?

Use 1-NN algorithm to predict

A	{Apple, Banana, Soymilk, Yoghurt}	Vegetarian
B	{Apple, Peanuts, Yoghurt}	Vegetarian
C	{Tomatoes, Potatoes, Yoghurt}	Meat-eater
D	{Apple, Tomatoes, Potatoes}	Meat-eater
E	{Apple, Tomatoes}	?=Meat-eater: class(D)
F	{Yoghurt}	?=both B,C are nearest neighbors, pick randomly

Jaccard similarity(E,A) = 1/5 = 0.2
Jaccard similarity(E,B) = 1/4 = 0.25
Jaccard similarity(E,C) = 1/4 = 0.25
Jaccard similarity(E,D) = 2/3 = 0.66

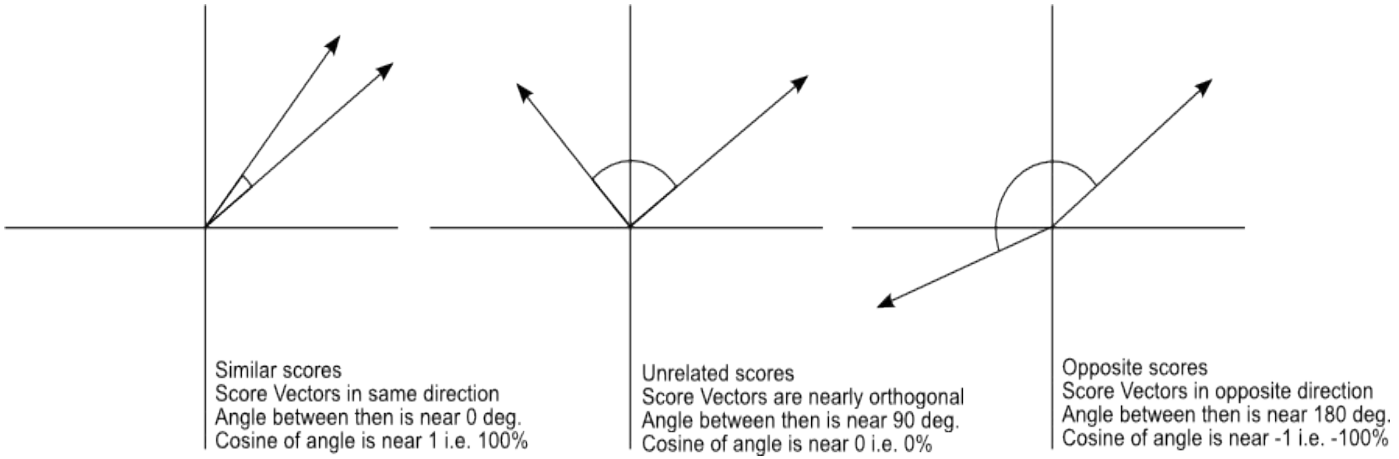
Jaccard similarity(F,A) = 1/4 = 0.25
Jaccard similarity(F,B) = 1/3 = 0.33
Jaccard similarity(F,C) = 1/3 = 0.33
Jaccard similarity(F,D) = 0/4 = 0

Cosine similarity

- Similarity between –1 and 1
- Between two real-valued vectors, *x* and *y*

$$\cos(\vec{x}, \vec{y}) = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \|\vec{y}\|}$$

Cosine similarity



Hamming distance

- Compare two strings
- Difference character-by-character of two strings
- Eg.: distance between “shoe” and “hose”
  - =?
- Strings can be DNA sequences
  - AGTCAA and GATCAA

Hamming distance

- Compare two strings
- Difference character-by-character of two strings
- Eg.: distance between “shoe” and “hose”
  - $s \neq h, h \neq o, o \neq s, e = e$
  - =3

Custom distance metrics

- If you have domain knowledge, then construct a distance metric specific to the domain
- E.g., know that dimension 1 is more important than dimension 2

Mentimeter



MidSemester Survey

Link: https://forms.gle/4SPwM15Vkh7s91847